EDA

Dataset Inspection

```
d <- read.csv("academy_final.csv")
str(d)
'data.frame':   201 obs. of  49 variables:
 $ film                                : chr  "Chocolat" "Crouching Tiger, Hidden Dragon" "Erin Brockovi
 $ release_year                        : int  2000 2000 2000 2000 2000 2001 2001 2001 2001 2001 ...
 $ director                            : chr  "Lasse Hallström" "Ang Lee" "Steven Soderbergh" "Ridley S
 $ best_picture_winner                 : int  0 0 0 1 0 1 0 0 0 0 ...
 $ tomatometer                         : int  63 96 87 80 93 74 87 93 75 91 ...
 $ popcornmeter                        : int  83 86 81 87 85 93 78 81 89 95 ...
 $ total_academy_nominations           : int  5 10 5 12 5 8 7 5 8 13 ...
 $ total_academy_award_exclude_best_pic: int  0 4 1 4 4 3 1 0 2 4 ...
 $ running_time_minutes                : int  121 120 130 155 147 135 137 131 128 178 ...
 $ genre                               : chr  "Drama, Romance" "Adventure, Action, Drama, Fantasy" "Dra
 $ rating                              : chr  "PG-13 (Some Violence|Scene of Sexuality)" "PG-13 (Some S
 $ language                            : chr  "English" "Chinese" "English" "English" ...
 $ budget_millions                     : num  25 17 52 103 48 58 19.8 1.7 50 93 ...
 $ box_office_millions                 : num  153 214 256 466 208 ...
 $ nominee_film_edit                   : int  0 1 0 1 1 1 0 0 0 1 ...
 $ award_film_edit                     : int  0 0 0 0 1 0 0 0 0 0 ...
 $ nominee_drama_gg                    : int  0 0 1 1 1 1 0 1 0 1 ...
 $ won_drama_gg                        : int  0 0 0 1 0 1 0 0 0 0 ...
 $ nominee_comedy_gg                   : int  1 0 0 0 0 0 1 0 1 0 ...
 $ won_comedy_gg                       : int  0 0 0 0 0 0 0 0 0 0 ...
 $ best_director_nominee_before        : int  2 0 0 1 0 0 4 0 0 0 ...
 $ best_director_nominee_include_now   : int  2 1 0 2 0 0 5 0 0 1 ...
 $ best_director_award_before          : int  0 0 0 0 0 0 0 0 0 0 ...
 $ best_director_award_now             : int  0 0 1 0 1 1 0 0 0 0 ...
 $ wikipedia                           : chr  "https://en.wikipedia.org/wiki/Chocolat_(2000_film)" "htt
 $ rotten_tomato                       : chr  "https://www.rottentomatoes.com/m/chocolat" "https://www.
 $ characters                          : int  8 30 15 9 7 16 12 14 13 49 ...
 $ words                               : int  1 4 2 1 1 3 2 3 2 10 ...
 $ genre_Drama                         : int  1 1 1 1 1 1 1 1 1 0 ...
 $ genre_Comedy                        : int  0 0 1 0 0 0 1 0 0 0 ...
 $ genre_History                       : int  0 0 0 1 0 0 0 0 0 0 ...
 $ genre_Biography                     : int  0 0 0 0 0 1 0 0 0 0 ...
 $ genre_Mystery___Thriller            : int  0 0 0 0 1 1 1 0 0 0 ...
 $ genre_Romance                       : int  1 0 1 0 0 1 0 0 1 0 ...
 $ genre_Adventure                     : int  0 1 0 1 0 0 0 0 0 1 ...
 $ genre_Action                        : int  0 1 0 1 1 0 0 0 0 0 ...
 $ genre_Crime                         : int  0 0 0 0 1 0 1 0 0 0 ...
 $ genre_Fantasy                       : int  0 1 0 0 0 0 0 0 0 1 ...
 $ genre_Sci_Fi                        : int  0 0 0 0 0 0 0 0 0 0 ...
 $ genre_War                           : int  0 0 0 0 0 0 0 0 0 0 ...
```

```
 $ genre_LGBTQ_                      : int  0 0 0 0 0 0 0 0 0 0 0 ...
 $ genre_Music                       : int  0 0 0 0 0 0 0 0 0 1 0 ...
 $ genre_Musical                     : int  0 0 0 0 0 0 0 0 0 1 0 ...
 $ genre_Western                     : int  0 0 0 0 0 0 0 0 0 0 0 ...
 $ genre_Horror                      : int  0 0 0 0 0 0 0 0 0 0 0 ...
 $ genre_Kids___Family               : int  0 0 0 0 0 0 0 0 0 0 0 ...
 $ genre_Animation                   : int  0 0 0 0 0 0 0 0 0 0 0 ...
 $ genre_Holiday                     : int  0 0 0 0 0 0 0 0 0 0 0 ...
 $ genre_Sports                      : int  0 0 0 0 0 0 0 0 0 0 0 ...
```

Based on the rough structure above, we have the following steps

(1) Dropping not useful variables. This include

- release_year
- director
- wikipedia
- rotten_tomato

(2) In addition, we should organize the following variables

- genres: Counting the number of genres using the variable, and then drop it
- rating: Organize them to be only PG13, R and G (There is a NA value)
- language: We should inspect the type of languages of the movies by category, drop it if most of them are English
- box-office: Observe distribution of the box-office to see if impacted significantly by the year
- genre_: The genre variables are too sparse. We can try doing two

```
# dropping columns
d <- d[, !names(d) %in% c("director", "wikipedia", "rotten_tomato")]
names(d)
 [1] "film"
 [2] "release_year"
 [3] "best_picture_winner"
 [4] "tomatometer"
 [5] "popcornmeter"
 [6] "total_academy_nominations"
 [7] "total_academy_award_exclude_best_pic"
 [8] "running_time_minutes"
 [9] "genre"
[10] "rating"
[11] "language"
[12] "budget_millions"
[13] "box_office_millions"
[14] "nominee_film_edit"
[15] "award_film_edit"
[16] "nominee_drama_gg"
[17] "won_drama_gg"
[18] "nominee_comedy_gg"
[19] "won_comedy_gg"
[20] "best_director_nominee_before"
[21] "best_director_nominee_include_now"
[22] "best_director_award_before"
[23] "best_director_award_now"
[24] "characters"
[25] "words"
```

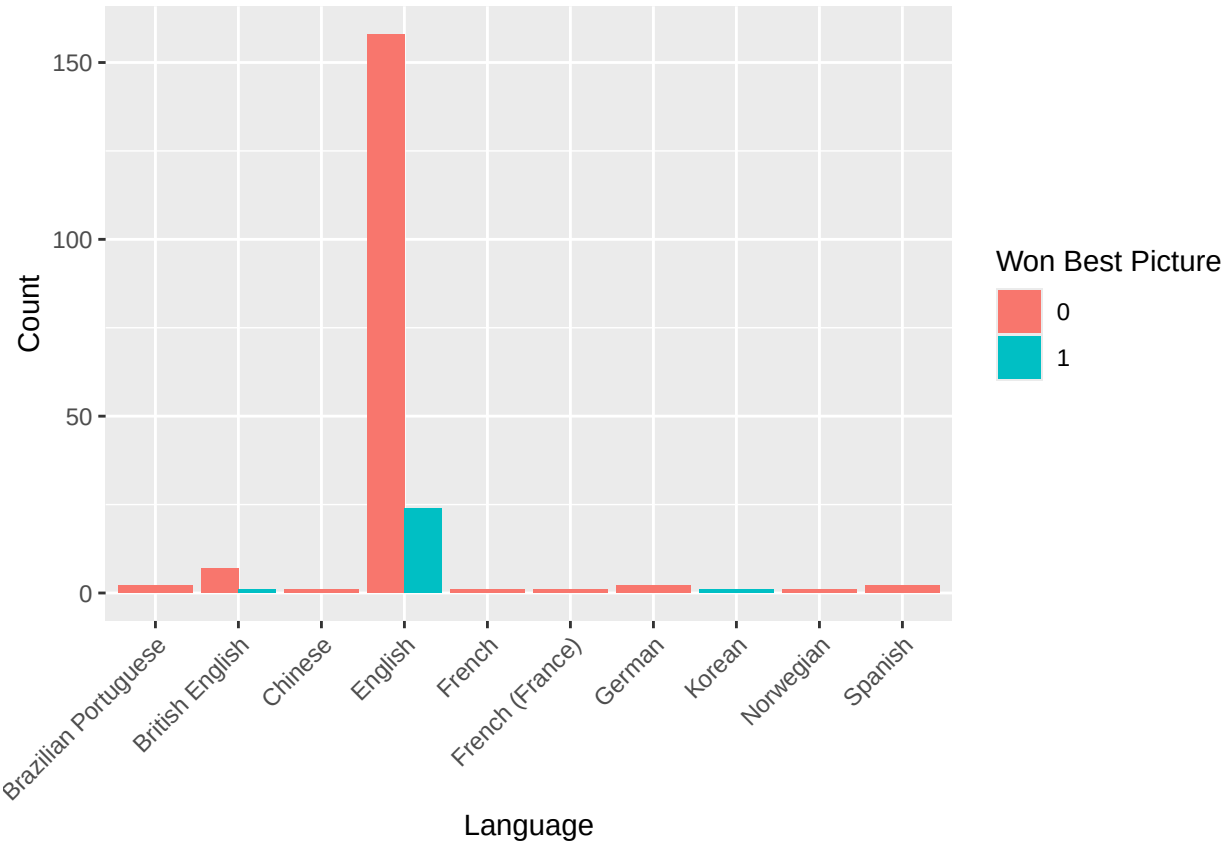
```
[26] "genre_Drama"
[27] "genre_Comedy"
[28] "genre_History"
[29] "genre_Biography"
[30] "genre_Mystery___Thriller"
[31] "genre_Romance"
[32] "genre_Adventure"
[33] "genre_Action"
[34] "genre_Crime"
[35] "genre_Fantasy"
[36] "genre_Sci_Fi"
[37] "genre_War"
[38] "genre_LGBTQ_"
[39] "genre_Music"
[40] "genre_Musical"
[41] "genre_Western"
[42] "genre_Horror"
[43] "genre_Kids___Family"
[44] "genre_Animation"
[45] "genre_Holiday"
[46] "genre_Sports"
```

```
# replace 'genre' column with genre count for each film
d$genre <- sapply(d$genre, function(g) length(unlist(strsplit(g, ", "))))
names(d)[names(d) == "genre"] <- "genre_count"
```

```
# extract base rating (e.g. 'PG-13', 'R', 'PG') from full rating string
d$rating <- gsub(" \\(.*", "", d$rating)
table(d$rating)

    G    PG PG-13     R
    1     9    76   114

# filling NA value
d[which(is.na(d$rating)), ]
            film release_year best_picture_winner tomatometer popcornmeter
155 Drive My Car         2021                   0          97           78
    total_academy_nominations total_academy_award_exclude_best_pic
155                         4                                   1
    running_time_minutes genre_count rating language budget_millions
155                  179           1   <NA>  English             1.3
    box_office_millions nominee_film_edit award_film_edit nominee_drama_gg
155                15.4                 0               0                0
    won_drama_gg nominee_comedy_gg won_comedy_gg best_director_nominee_before
155            0                 0             0                            0
    best_director_nominee_include_now best_director_award_before
155                                 0                         0
    best_director_award_now characters words genre_Drama genre_Comedy
155                       0         12     3           1            0
    genre_History genre_Biography genre_Mystery___Thriller genre_Romance
155             0               0                        0             0
    genre_Adventure genre_Action genre_Crime genre_Fantasy genre_Sci_Fi
155               0            0           0             0            0
    genre_War genre_LGBTQ_ genre_Music genre_Musical genre_Western genre_Horror
```

```
155         0            0           0             0             0            0
    genre_Kids___Family genre_Animation genre_Holiday genre_Sports
155                   0               0             0            0
# Film 'Drive My Car' is 'not rated', we classify it as R
d[which(is.na(d$rating)), ]$rating <- "R"
table(d$rating)

    G    PG PG-13     R
    1     9    76   115
```

```
# inspect film language distribution
table(d$language)

Brazilian Portuguese      British English              Chinese
                   2                    8                    1
             English               French      French (France)
                 182                    1                    1
              German               Korean            Norwegian
                   2                    1                    1
             Spanish
                   2
table(d$language, d$best_picture_winner)

                        0   1
  Brazilian Portuguese  2   0
  British English       7   1
  Chinese               1   0
  English             158  24
  French                1   0
  French (France)       1   0
  German                2   0
  Korean                0   1
  Norwegian             1   0
  Spanish               2   0

library(ggplot2)
Warning: package 'ggplot2' was built under R version 4.3.3

ggplot(d, aes(x = language, fill = factor(best_picture_winner))) + geom_bar(position = "dodge") +
    labs(x = "Language", y = "Count", fill = "Won Best Picture") + theme(axis.text.x = element_text(angl
    hjust = 1))
```

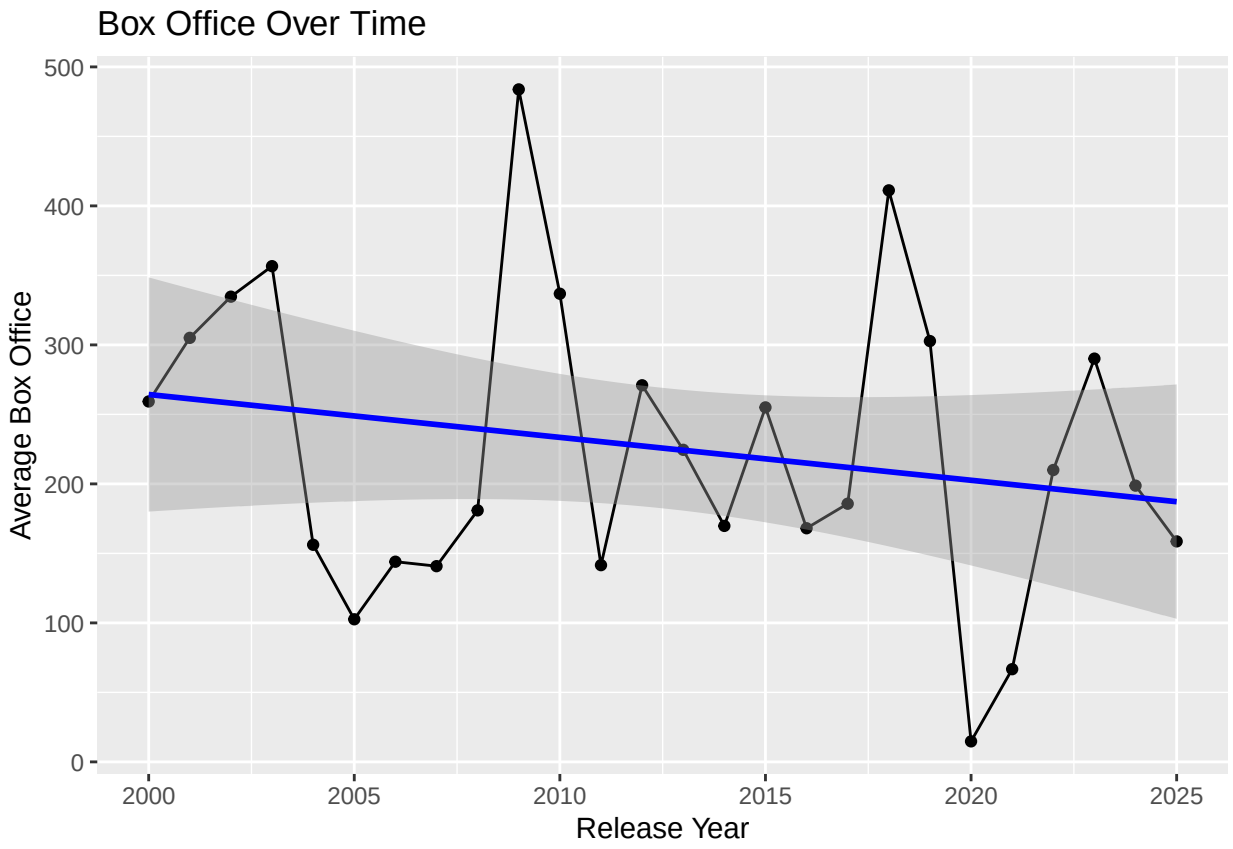


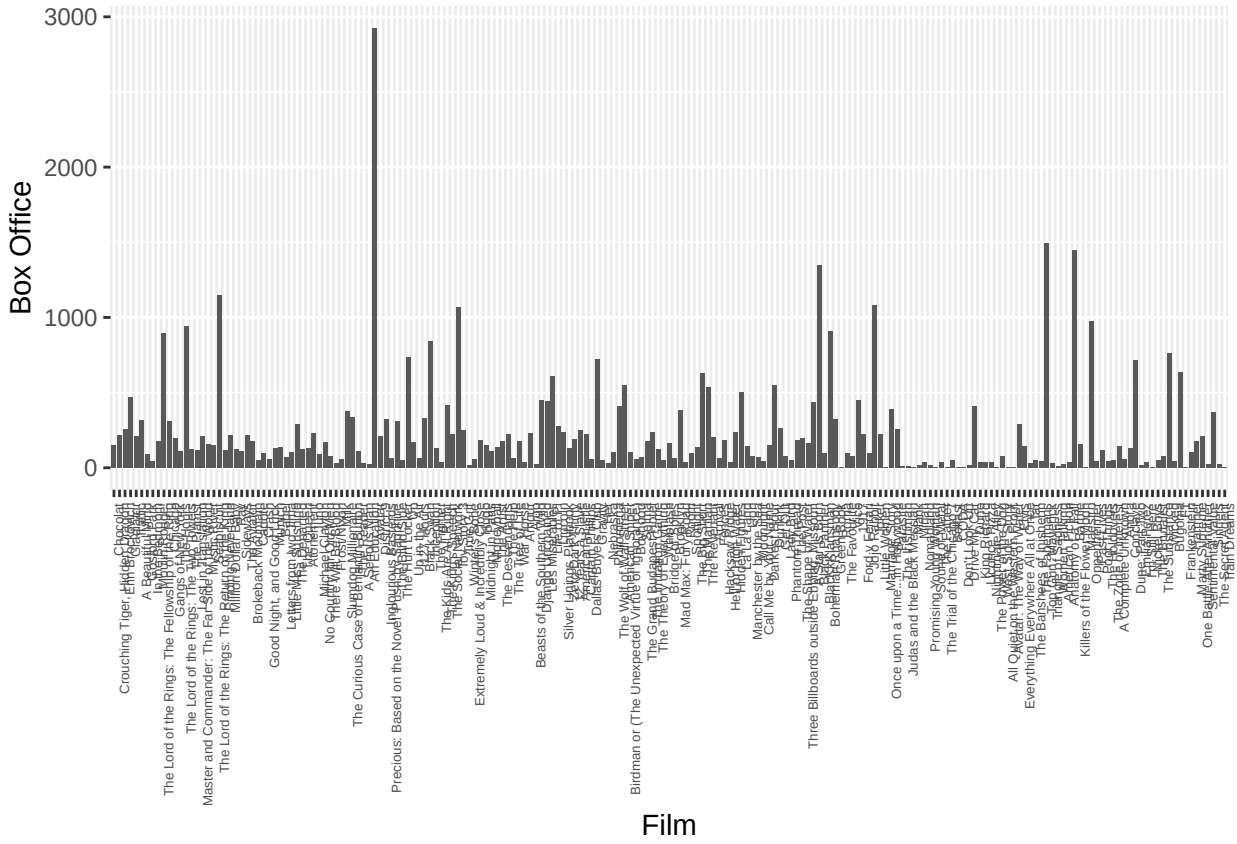

```
# low correlation/pattern -> drop language
d <- d[, !names(d) %in% c("language")]
```

```
# normalize the box office for the Netflix film (All Quiet on the Western Front
# (2022))
d[d$box_office_millions == 2878, ]$box_office_millions = 2878/1e+06

names(d)
 [1] "film"
 [2] "release_year"
 [3] "best_picture_winner"
 [4] "tomatometer"
 [5] "popcornmeter"
 [6] "total_academy_nominations"
 [7] "total_academy_award_exclude_best_pic"
 [8] "running_time_minutes"
 [9] "genre_count"
[10] "rating"
[11] "budget_millions"
[12] "box_office_millions"
[13] "nominee_film_edit"
[14] "award_film_edit"
[15] "nominee_drama_gg"
[16] "won_drama_gg"
[17] "nominee_comedy_gg"
[18] "won_comedy_gg"
[19] "best_director_nominee_before"
```

```
[20] "best_director_nominee_include_now"
[21] "best_director_award_before"
[22] "best_director_award_now"
[23] "characters"
[24] "words"
[25] "genre_Drama"
[26] "genre_Comedy"
[27] "genre_History"
[28] "genre_Biography"
[29] "genre_Mystery___Thriller"
[30] "genre_Romance"
[31] "genre_Adventure"
[32] "genre_Action"
[33] "genre_Crime"
[34] "genre_Fantasy"
[35] "genre_Sci_Fi"
[36] "genre_War"
[37] "genre_LGBTQ_"
[38] "genre_Music"
[39] "genre_Musical"
[40] "genre_Western"
[41] "genre_Horror"
[42] "genre_Kids___Family"
[43] "genre_Animation"
[44] "genre_Holiday"
[45] "genre_Sports"
plot_box_office <- function(df, box_col = "box_office_millions") {

    # aggregate average box office by year
    box_by_year <- aggregate(df[[box_col]] ~ df$release_year, data = df, FUN = mean,
        na.rm = TRUE)
    colnames(box_by_year) <- c("release_year", box_col)

    # trend line
    print(ggplot(box_by_year, aes(x = release_year, y = .data[[box_col]])) + geom_line() +
        geom_point() + geom_smooth(method = "lm", se = TRUE, color = "blue") + labs(x = "Release Year",
        y = "Average Box Office", title = "Box Office Over Time"))

    # overall bar chart ordered by year
    print(ggplot(df, aes(x = reorder(film, release_year), y = .data[[box_col]])) +
        geom_bar(stat = "identity") + labs(x = "Film", y = "Box Office") + theme(axis.text.x = element_
        hjust = 1, size = 5)))

    # interactive stacked bar chart
    p <- ggplot(df, aes(x = factor(release_year), y = .data[[box_col]], fill = reorder(film,
        .data[[box_col]]), text = paste("Film:", film, "<br>Box Office:", .data[[box_col]]))) +
        geom_bar(stat = "identity") + labs(x = "Year", y = "Box Office") + theme(axis.text.x = element_
        hjust = 1), legend.position = "none")

    print(ggplotly(p, tooltip = "text"))
}

# call with raw
```

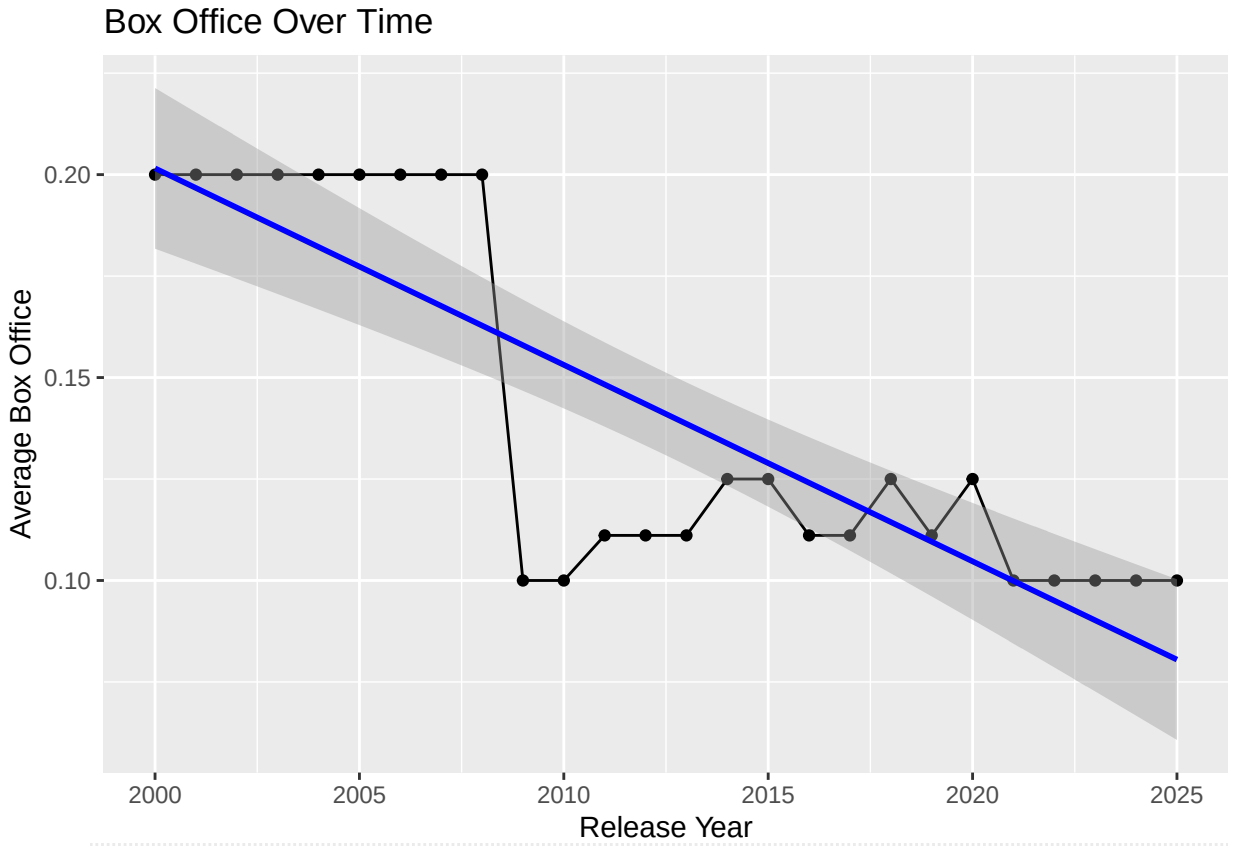
```
plot_box_office(d)
`geom_smooth()` using formula = 'y ~ x'
```

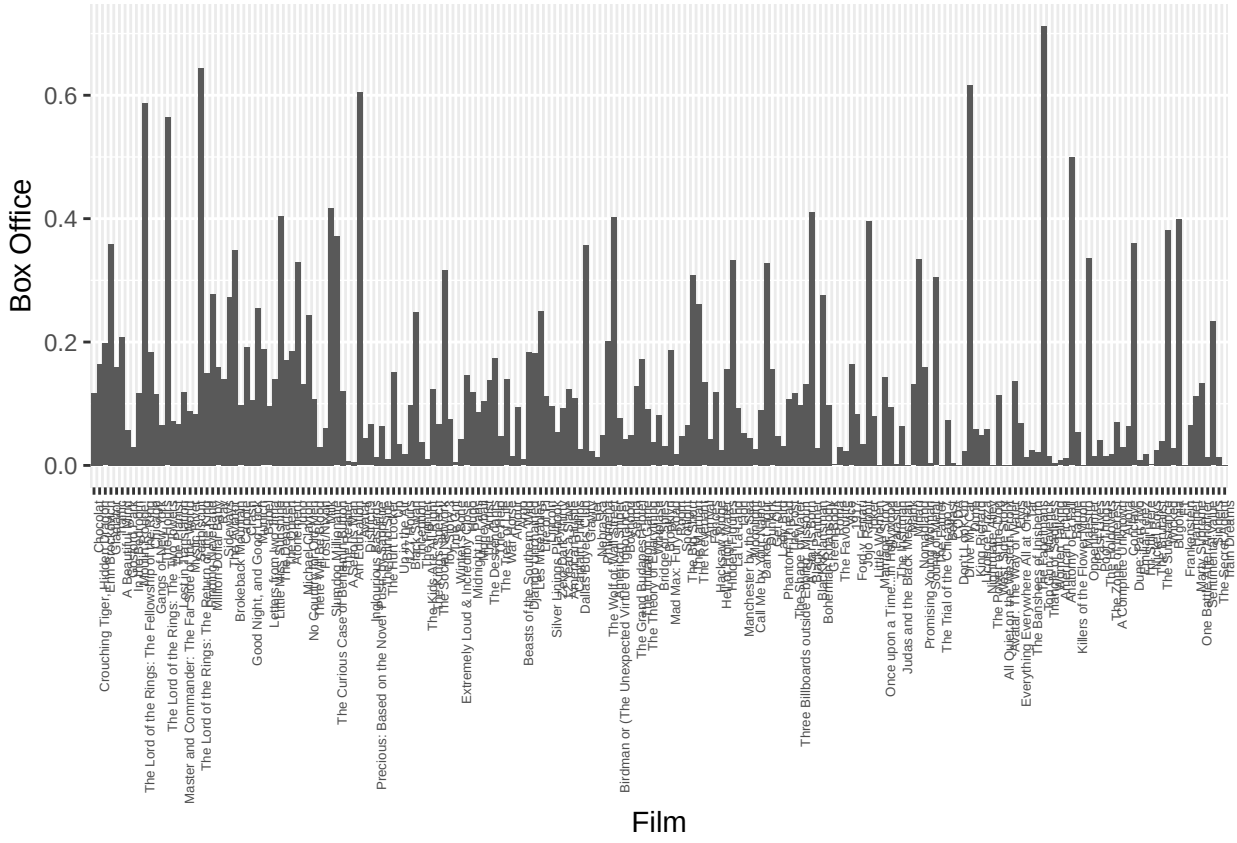





```
Error in ggplotly(p, tooltip = "text"): could not find function "ggplotly"
```

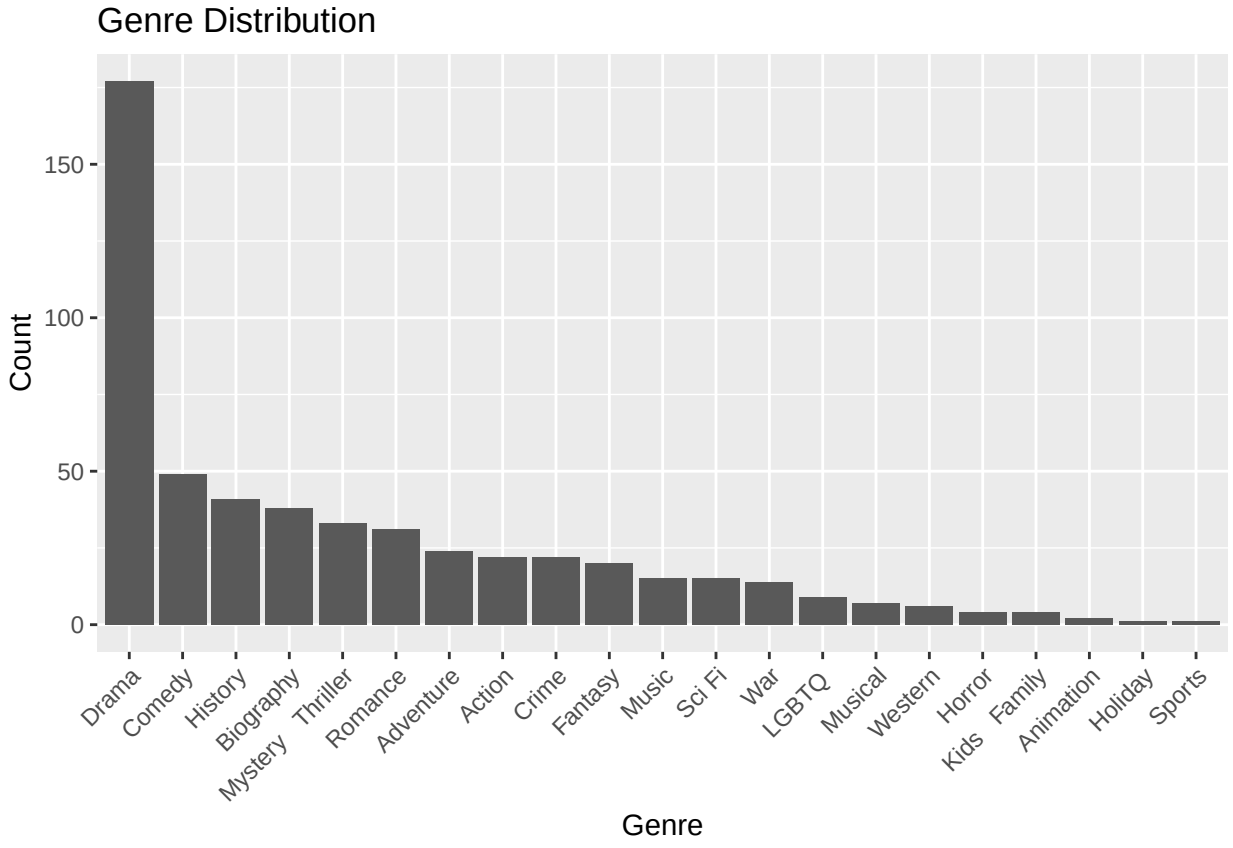
```
# normalize the box office by year~~~
library(dplyr)

Attaching package: 'dplyr'
The following objects are masked from 'package:stats':

    filter, lag
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

d <- d %>%
    group_by(release_year) %>%
    mutate(box_office_norm = box_office_millions/sum(box_office_millions, na.rm = TRUE)) %>%
    ungroup()

# call with normalized
plot_box_office(d, box_col = "box_office_norm")
`geom_smooth()` using formula = 'y ~ x'
```



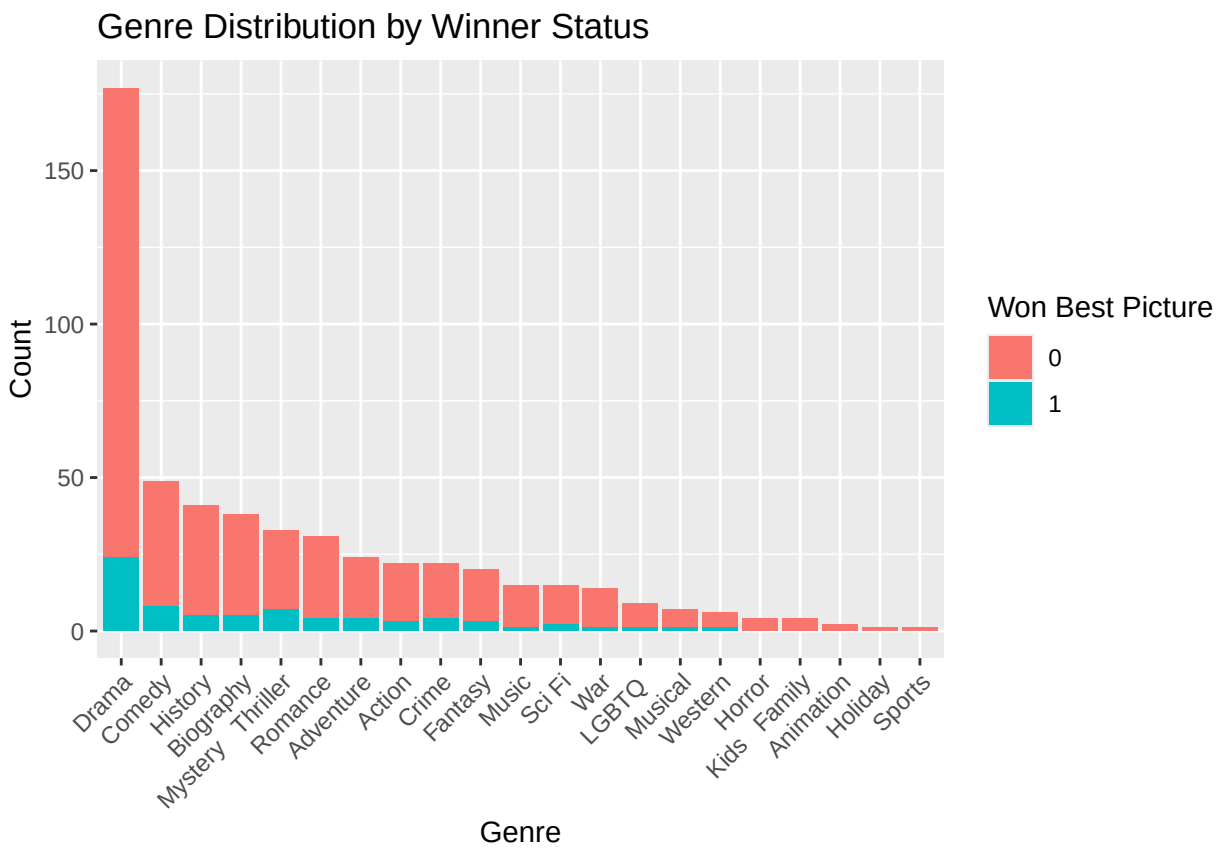



```
Error in ggplotly(p, tooltip = "text"): could not find function "ggplotly"
```

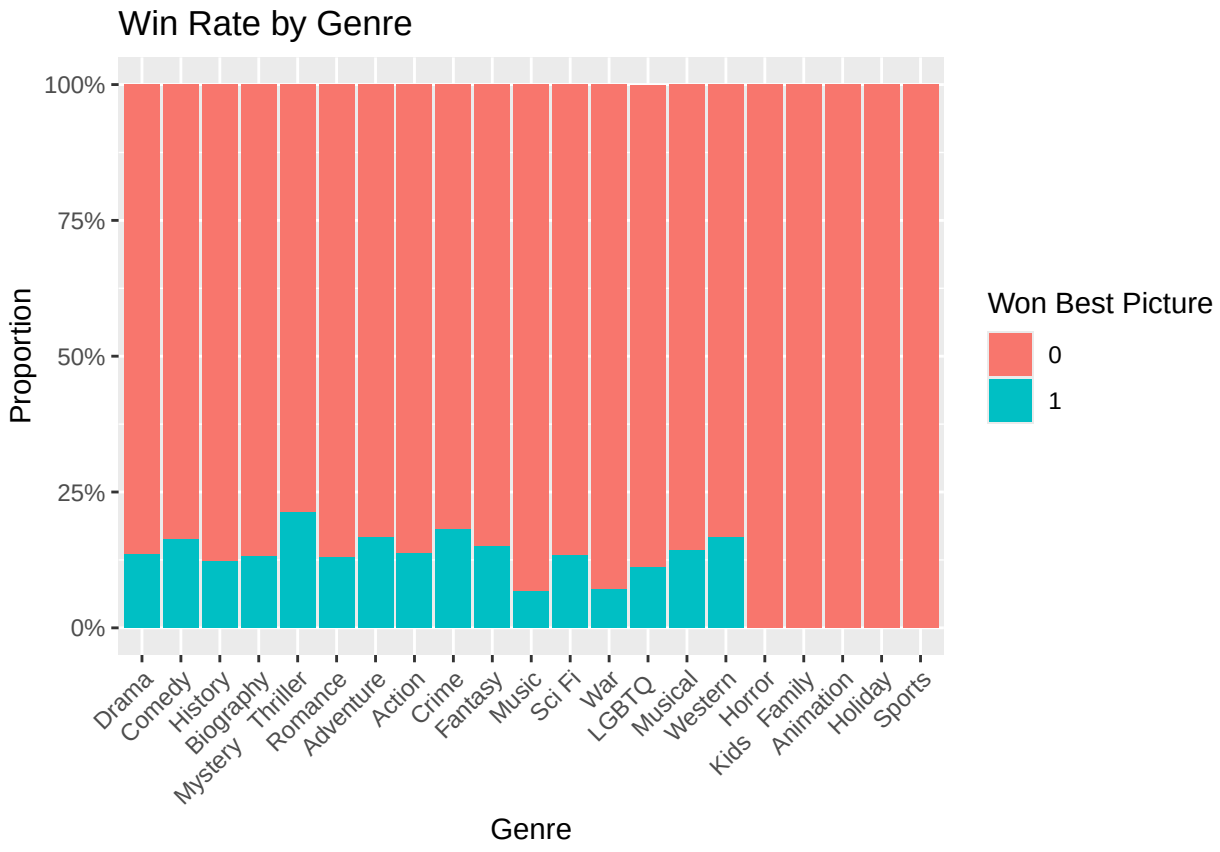
```r
# histograms for genres

library(tidyr)
# grab all genre indicator columns
genre_cols <- names(d)[grepl("^genre_", names(d)) & names(d) != "genre_count"]

# pivot to long format for easy plotting
genre_long <- d %>%
  select(film, best_picture_winner, all_of(genre_cols)) %>%
  pivot_longer(cols = all_of(genre_cols), names_to = "genre", values_to = "has_genre") %>%
  filter(has_genre == 1) %>%
  mutate(genre = gsub("genre_", "", genre),    # clean up prefix
         genre = gsub("_", " ", genre))         # underscores -> spaces

# simple genre frequency histogram
ggplot(genre_long, aes(x = reorder(genre, -table(genre)[genre]))) +
  geom_bar() +
  labs(x = "Genre", y = "Count", title = "Genre Distribution") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



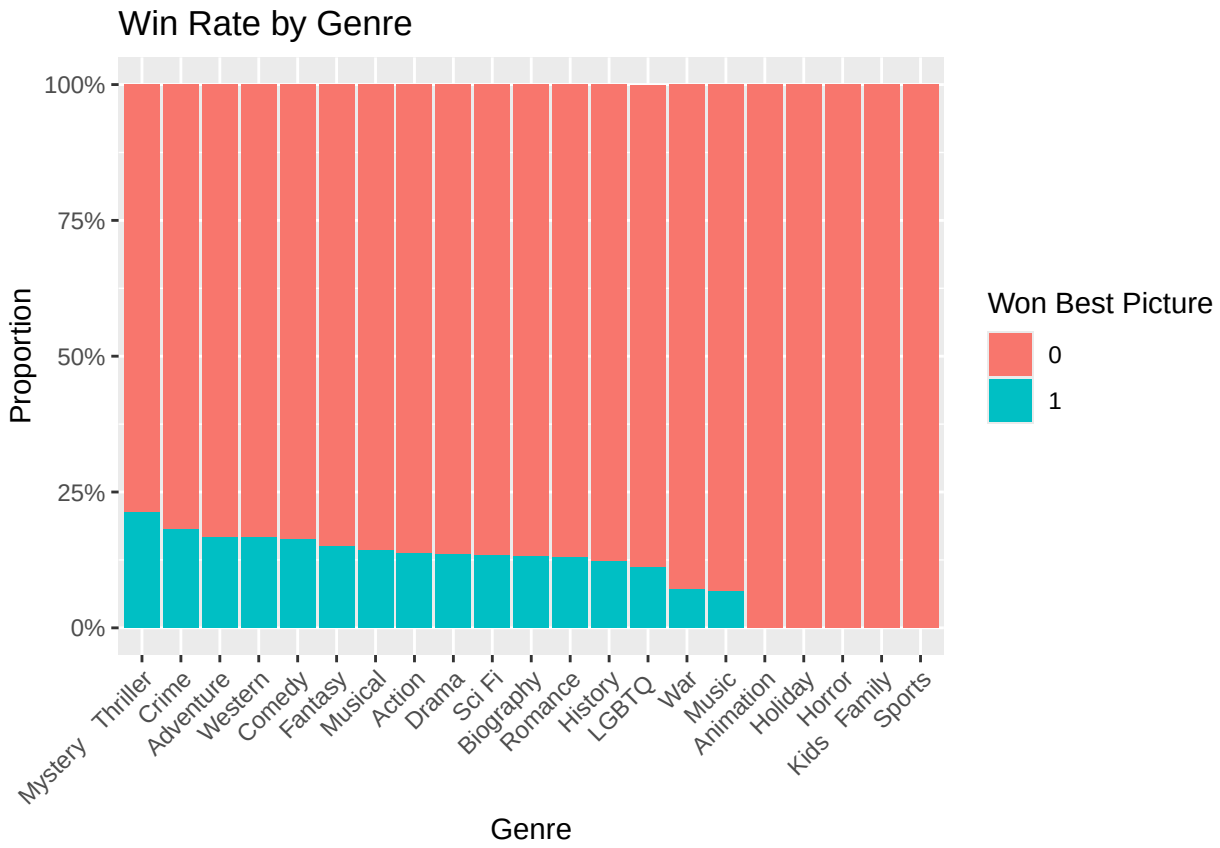

```r
# genre histogram stacked by winner status
ggplot(genre_long, aes(x = reorder(genre, -table(genre)[genre]),
                       fill = factor(best_picture_winner))) +
  geom_bar(position = "stack") +
  labs(x = "Genre", y = "Count", fill = "Won Best Picture",
       title = "Genre Distribution by Winner Status") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



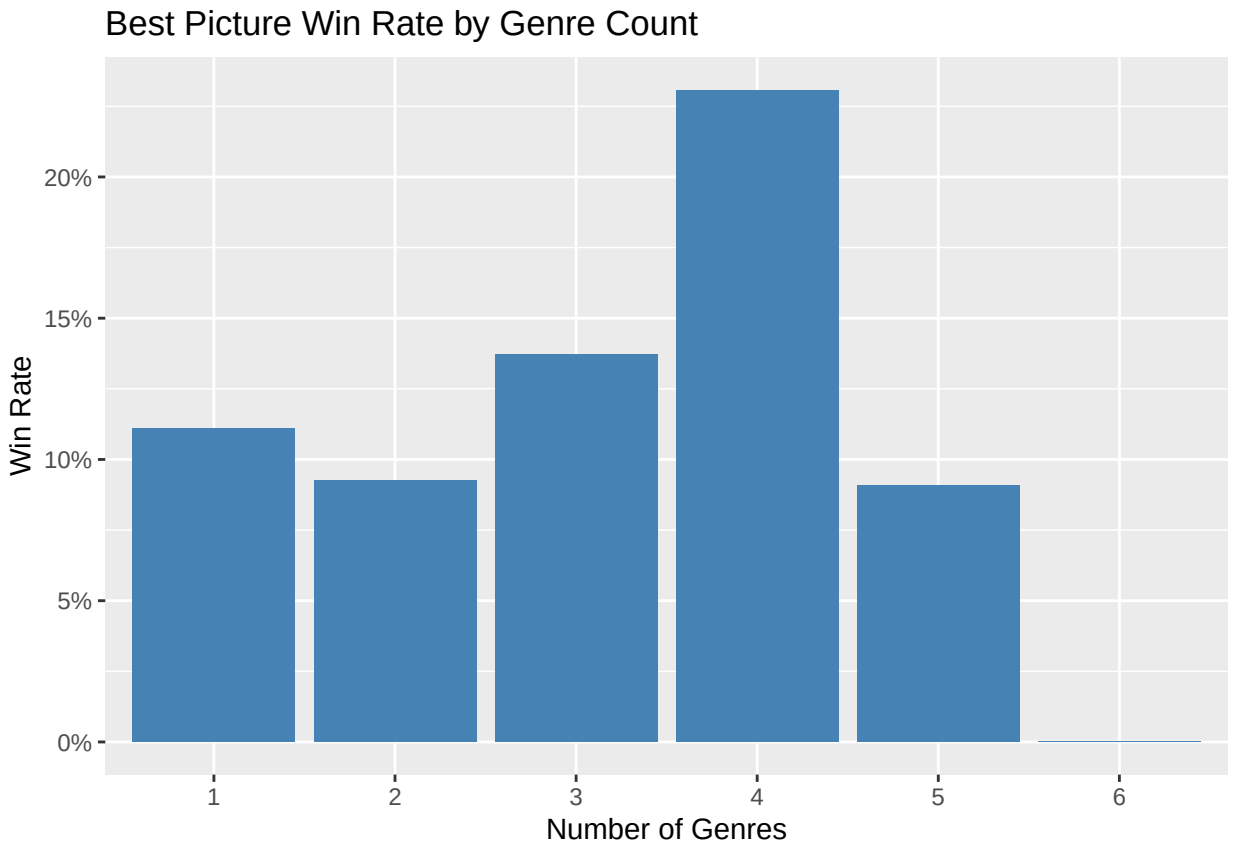

```r
# percentage of winner within genre
ggplot(genre_long, aes(x = reorder(genre, -table(genre)[genre]),
                       fill = factor(best_picture_winner))) +
  geom_bar(position = "fill") +
  labs(x = "Genre", y = "Proportion", fill = "Won Best Picture",
       title = "Win Rate by Genre") +
  scale_y_continuous(labels = scales::percent) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
# calculate win rate per genre to use for ordering
win_rate <- genre_long %>%
  group_by(genre) %>%
  summarise(win_rate = mean(best_picture_winner == 1)) %>%
  arrange(desc(win_rate))

ggplot(genre_long, aes(x = reorder(genre, -win_rate$win_rate[match(genre, win_rate$genre)]),
                       fill = factor(best_picture_winner))) +
  geom_bar(position = "fill") +
  labs(x = "Genre", y = "Proportion", fill = "Won Best Picture",
       title = "Win Rate by Genre") +
  scale_y_continuous(labels = scales::percent) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

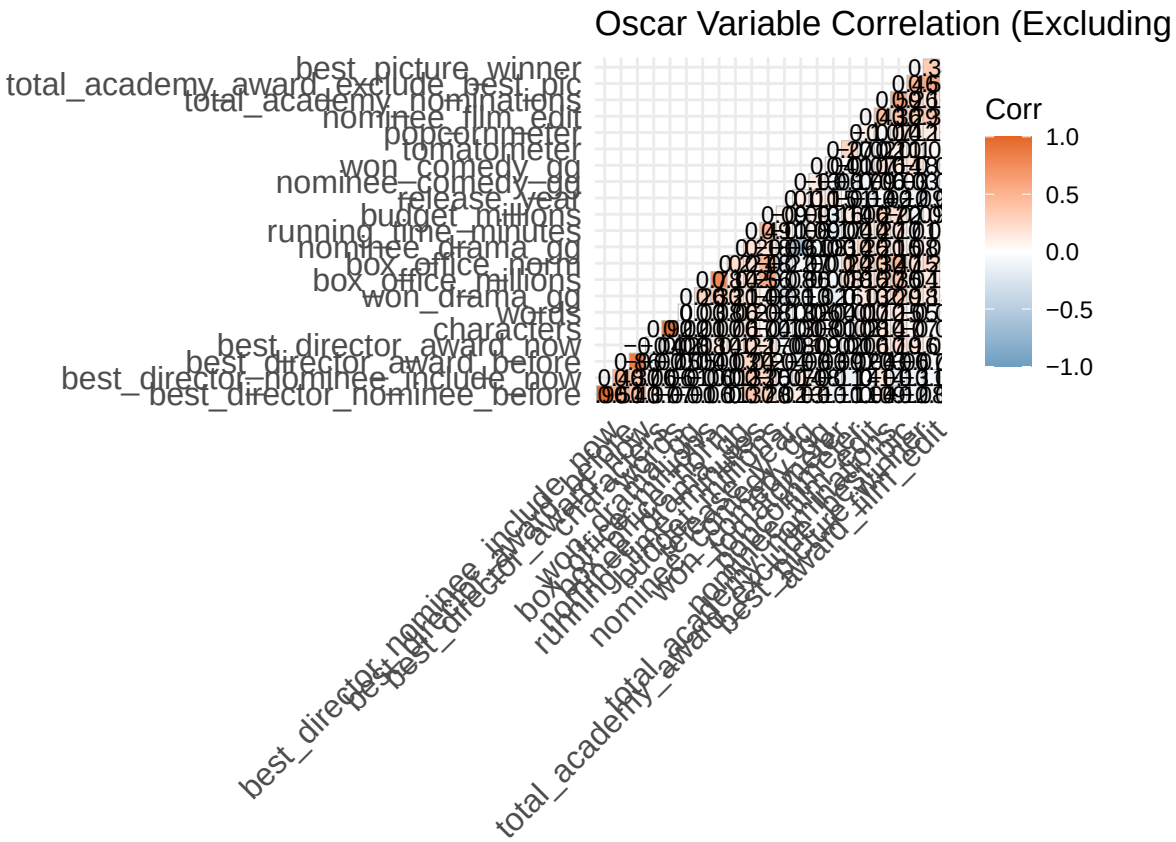



```
# genre count and winning rate distribution
table(d$genre_count)

 1  2  3  4  5  6
36 54 73 26 11  1

table(d[d$best_picture_winner == 1, ]$genre_count)

 1  2  3  4  5
 4  5 10  6  1
d %>%
    group_by(genre_count) %>%
    summarise(win_rate = mean(best_picture_winner == 1, na.rm = TRUE)) %>%
    ggplot(aes(x = factor(genre_count), y = win_rate)) + geom_bar(stat = "identity",
    fill = "steelblue") + scale_y_continuous(labels = scales::percent) + labs(x = "Number of Genres",
    y = "Win Rate", title = "Best Picture Win Rate by Genre Count")
```



Correlation Map

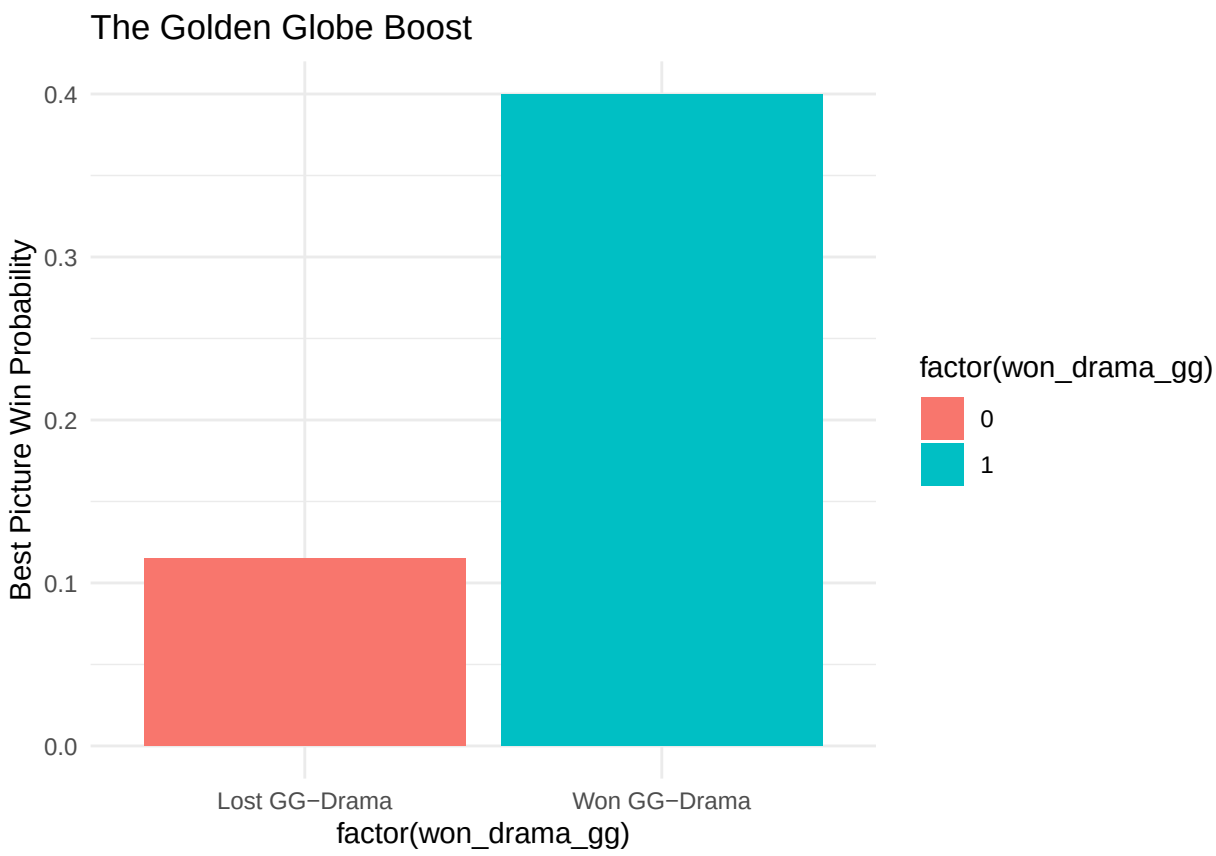
```
library(ggplot2)
library(dplyr)
library(ggcorrplot)

# 1. Filter for numeric variables AND exclude all genres This keeps the 'clean'
# numerical/award variables
numeric_data_no_genres <- d %>%
    select(where(is.numeric)) %>%
    select(-starts_with("genre_"))

# 2. Compute Correlation
corr_matrix <- cor(numeric_data_no_genres, use = "complete.obs")

# 3. Plot Heatmap Since we have fewer variables now, we can set lab = TRUE to
# see the exact coefficients
ggcorrplot(corr_matrix, hc.order = TRUE, type = "lower", lab = TRUE, lab_size = 3,
    title = "Oscar Variable Correlation (Excluding Genres)", colors = c("#6D9EC1",
        "white", "#E46726"))
```



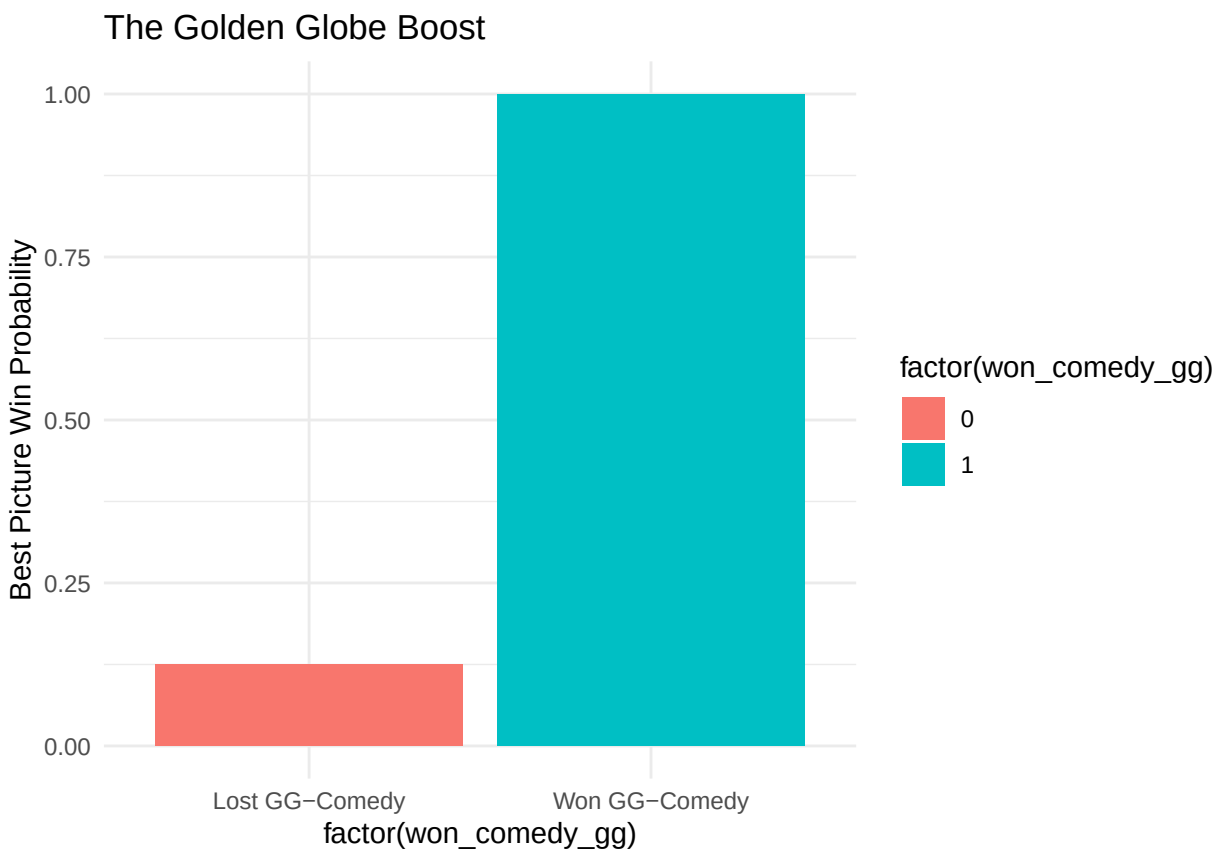
#

Does winning GG convert to winning Oscar Best Picture

```
par(mfrow = c(1, 2))

# A. Golden Globe Momentum Does a GG win 'lock in' an Oscar?
d %>%
    group_by(won_drama_gg) %>%
    summarise(win_rate = mean(best_picture_winner)) %>%
    ggplot(aes(x = factor(won_drama_gg), y = win_rate, fill = factor(won_drama_gg))) +
    geom_col() + scale_x_discrete(labels = c("Lost GG-Drama", "Won GG-Drama")) +
    labs(title = "The Golden Globe Boost", y = "Best Picture Win Probability") +
    theme_minimal()
```




```
d %>%
    group_by(won_comedy_gg) %>%
    summarise(win_rate = mean(best_picture_winner)) %>%
    ggplot(aes(x = factor(won_comedy_gg), y = win_rate, fill = factor(won_comedy_gg))) +
    geom_col() + scale_x_discrete(labels = c("Lost GG-Comedy", "Won GG-Comedy")) +
    labs(title = "The Golden Globe Boost", y = "Best Picture Win Probability") +
    theme_minimal()
```

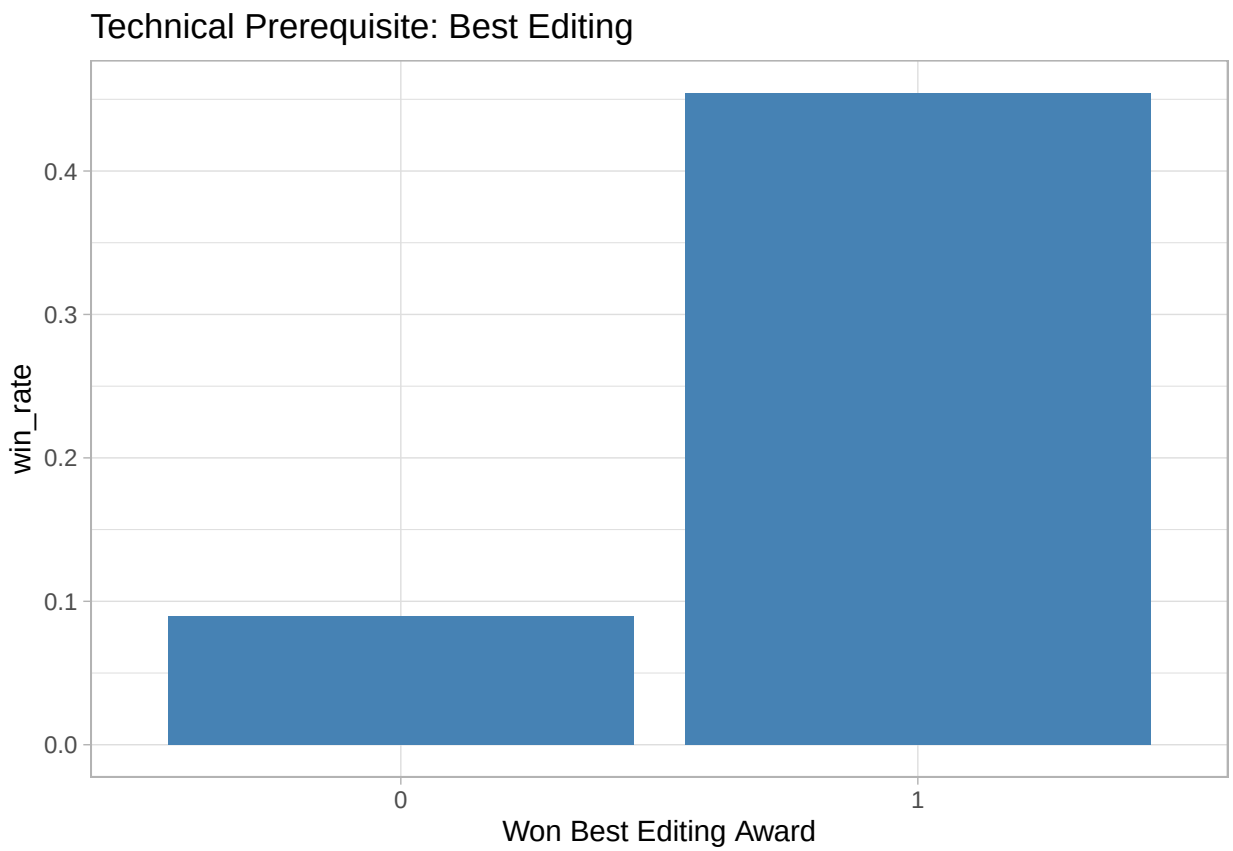



```
library(dplyr)
library(tidyr)

# Create a summary for Drama
drama_summary <- d %>%
  group_by(won_drama_gg) %>%
  summarise(
    Total_Films = n(),
    Oscar_Wins = sum(best_picture_winner),
    Win_Rate = mean(best_picture_winner)
  ) %>%
  mutate(Category = "Drama") %>%
  rename(Won_GG = won_drama_gg)

# Create a summary for Comedy
comedy_summary <- d %>%
  group_by(won_comedy_gg) %>%
  summarise(
    Total_Films = n(),
    Oscar_Wins = sum(best_picture_winner),
    Win_Rate = mean(best_picture_winner)
  ) %>%
  mutate(Category = "Comedy") %>%
  rename(Won_GG = won_comedy_gg)

# Combine them into one professional table
gg_conversion_table <- bind_rows(drama_summary, comedy_summary) %>%
```

```
  filter(Won_GG == 1) %>% # We only care about the films that actually won the GG
  select(Category, Total_Films, Oscar_Wins, Win_Rate) %>%
  mutate(Win_Rate = scales::percent(Win_Rate, accuracy = 0.1))

print(gg_conversion_table)
# A tibble: 2 x 4
  Category Total_Films Oscar_Wins Win_Rate
  <chr>          <int>      <int> <chr>
1 Drama             10          4 40.0%
2 Comedy             1          1 100.0%
```

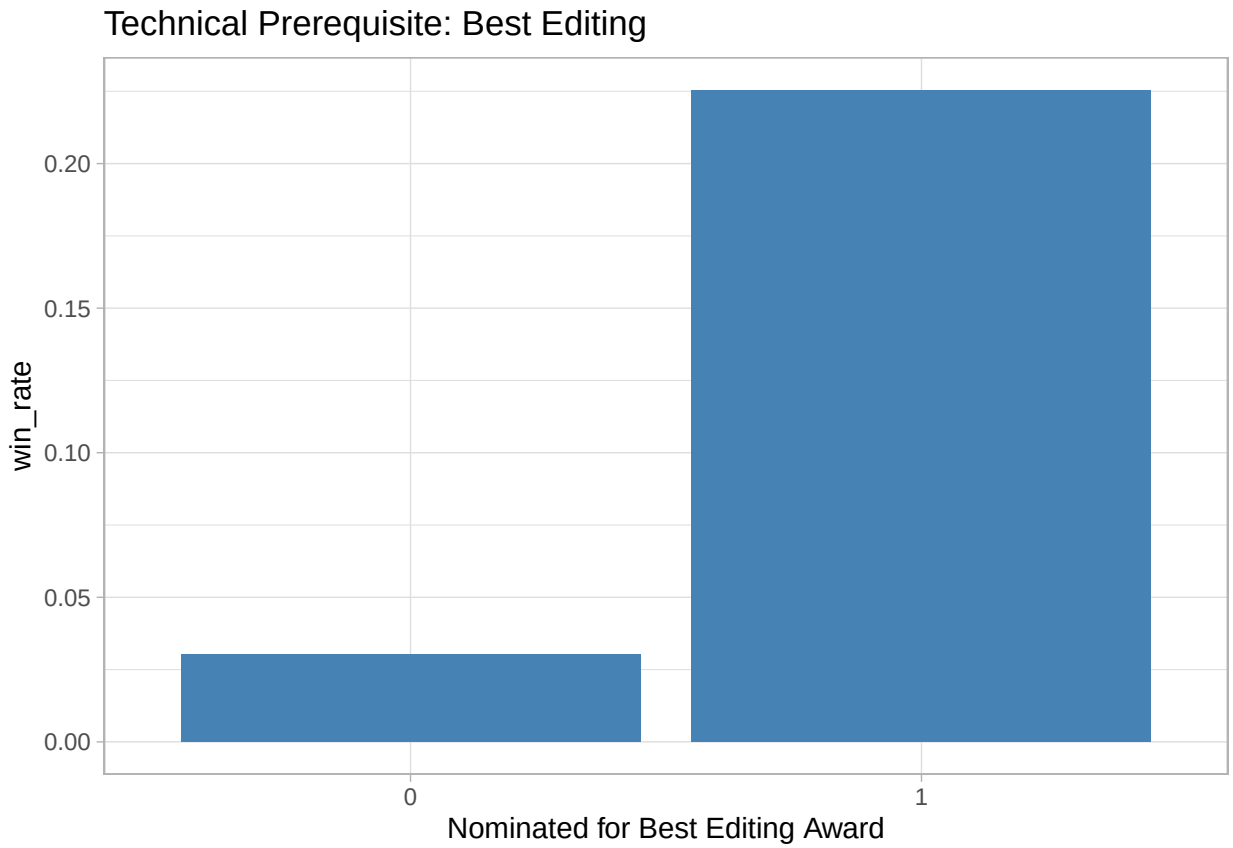
Does winning for best editing film convert to winning Oscar Best Picture

```
# B. The 'Best Editing' Rule Analyzing if the film_edit award is a prerequisite
d %>%
    group_by(award_film_edit) %>%
    summarise(win_rate = mean(best_picture_winner)) %>%
    ggplot(aes(x = factor(award_film_edit), y = win_rate)) + geom_bar(stat = "identity",
    fill = "steelblue") + labs(title = "Technical Prerequisite: Best Editing", x = "Won Best Editing Aw
    theme_light()
```



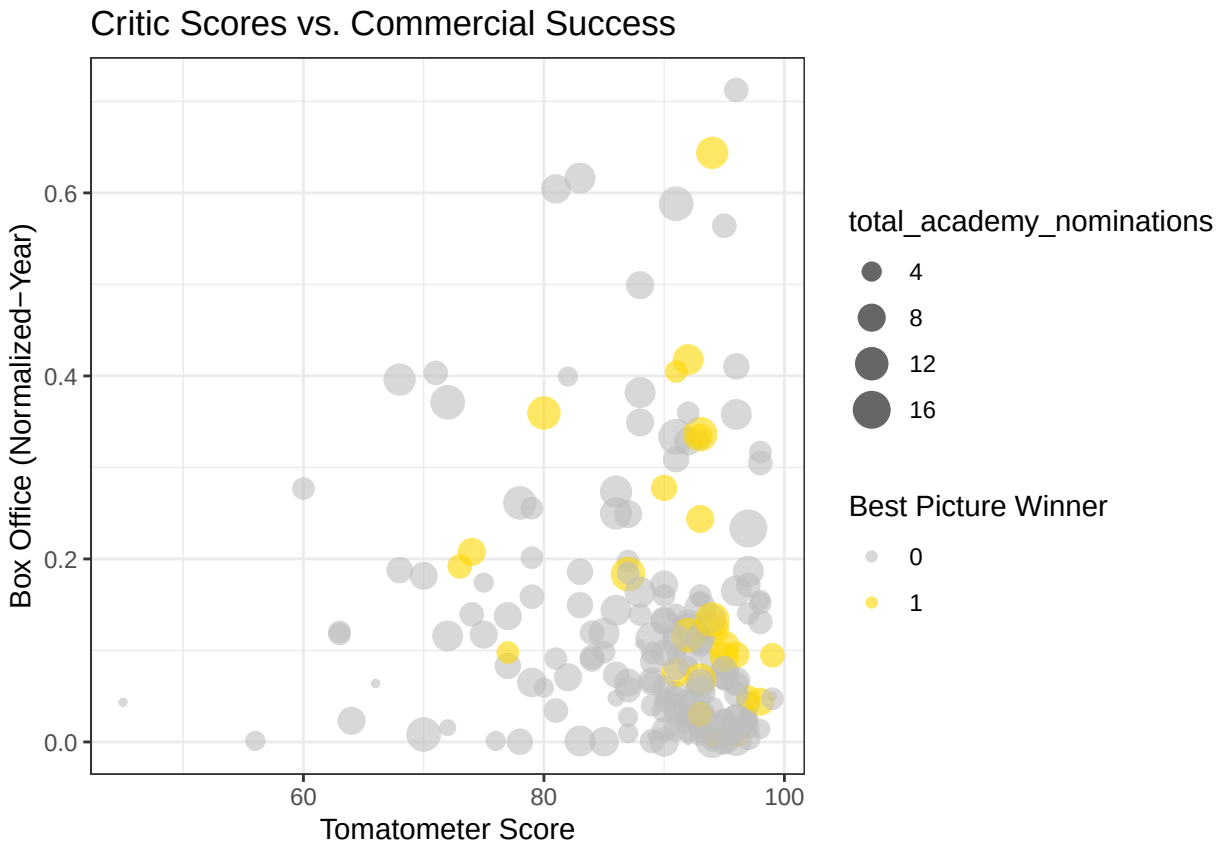

```
# B. The 'Best Editing' Rule Analyzing if the film_edit nominee is a
# prerequisite
d %>%
```

```
    group_by(nominee_film_edit) %>%
    summarise(win_rate = mean(best_picture_winner)) %>%
    ggplot(aes(x = factor(nominee_film_edit), y = win_rate)) + geom_bar(stat = "identity",
    fill = "steelblue") + labs(title = "Technical Prerequisite: Best Editing", x = "Nominated for Best
    theme_light()
```

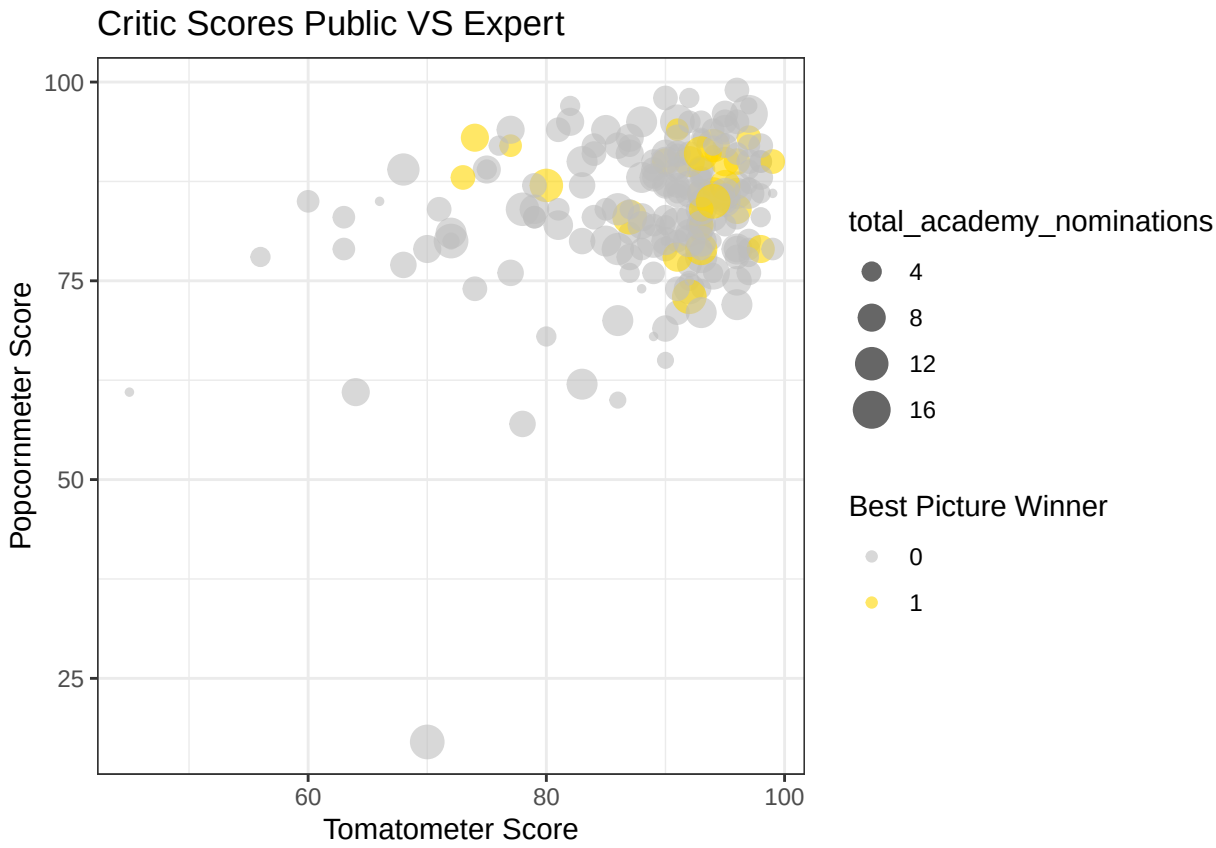


The distribution of winners in box office and tomotometer

```
# C. Tomatometer vs Box Office
ggplot(d, aes(x = tomatometer, y = box_office_norm, color = factor(best_picture_winner))) +
    geom_point(aes(size = total_academy_nominations), alpha = 0.6) + scale_color_manual(values = c("grey
    "gold"), name = "Best Picture Winner") + labs(title = "Critic Scores vs. Commercial Success",
    x = "Tomatometer Score", y = "Box Office (Normalized-Year)") + theme_bw()
```

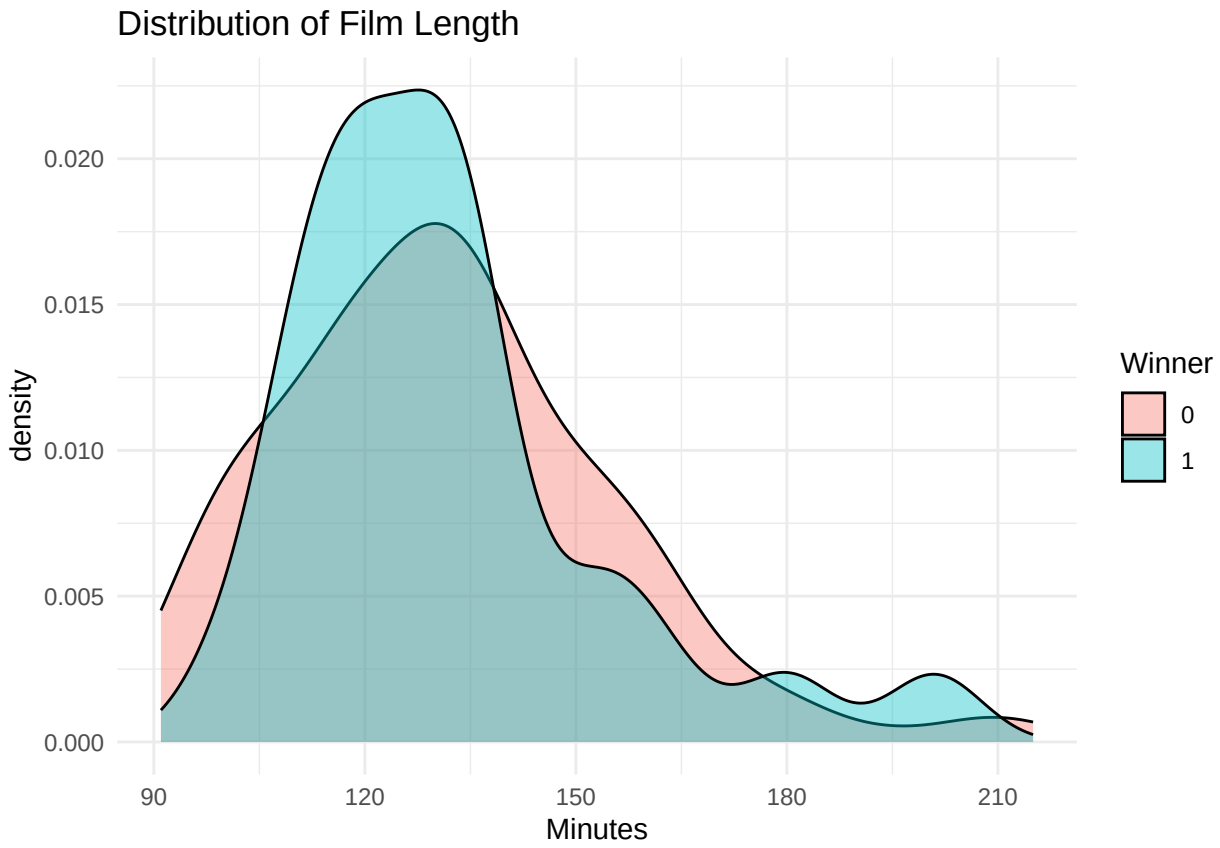



```
# C. Tomatometer vs Box Office
ggplot(d, aes(x = tomatometer, y = popcornmeter, color = factor(best_picture_winner))) +
    geom_point(aes(size = total_academy_nominations), alpha = 0.6) + scale_color_manual(values = c("grey
    "gold"), name = "Best Picture Winner") + labs(title = "Critic Scores Public VS Expert",
    x = "Tomatometer Score", y = "Popcornmeter Score") + theme_bw()
```

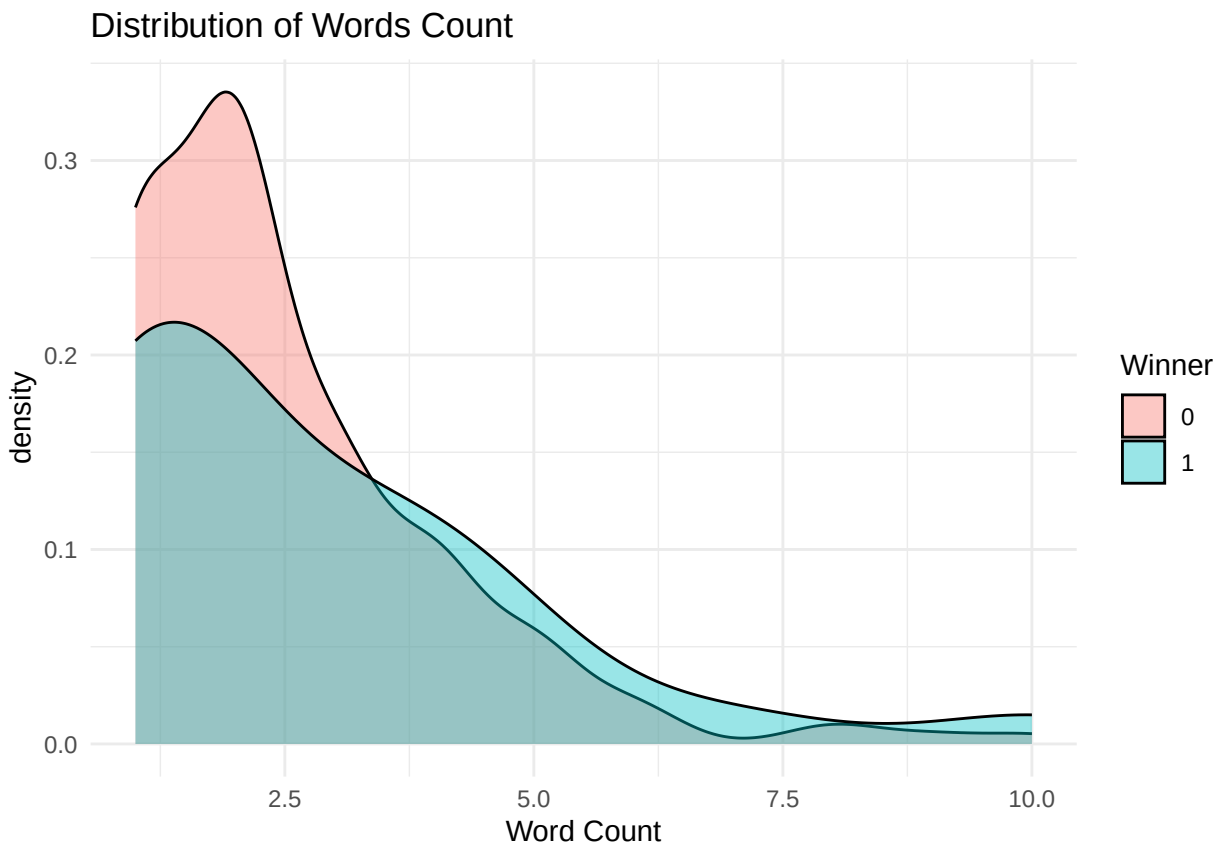



```
d[d$popcornmeter < 25, ]
# A tibble: 1 x 46
  film         release_year best_picture_winner tomatometer popcornmeter
  <chr>               <int>               <int>       <int>        <int>
1 Emilia Pérez         2024                   0          70           17
# i 41 more variables: total_academy_nominations <int>,
#   total_academy_award_exclude_best_pic <int>, running_time_minutes <int>,
#   genre_count <int>, rating <chr>, budget_millions <dbl>,
#   box_office_millions <dbl>, nominee_film_edit <int>, award_film_edit <int>,
#   nominee_drama_gg <int>, won_drama_gg <int>, nominee_comedy_gg <int>,
#   won_comedy_gg <int>, best_director_nominee_before <int>,
#   best_director_nominee_include_now <int>, ...
```

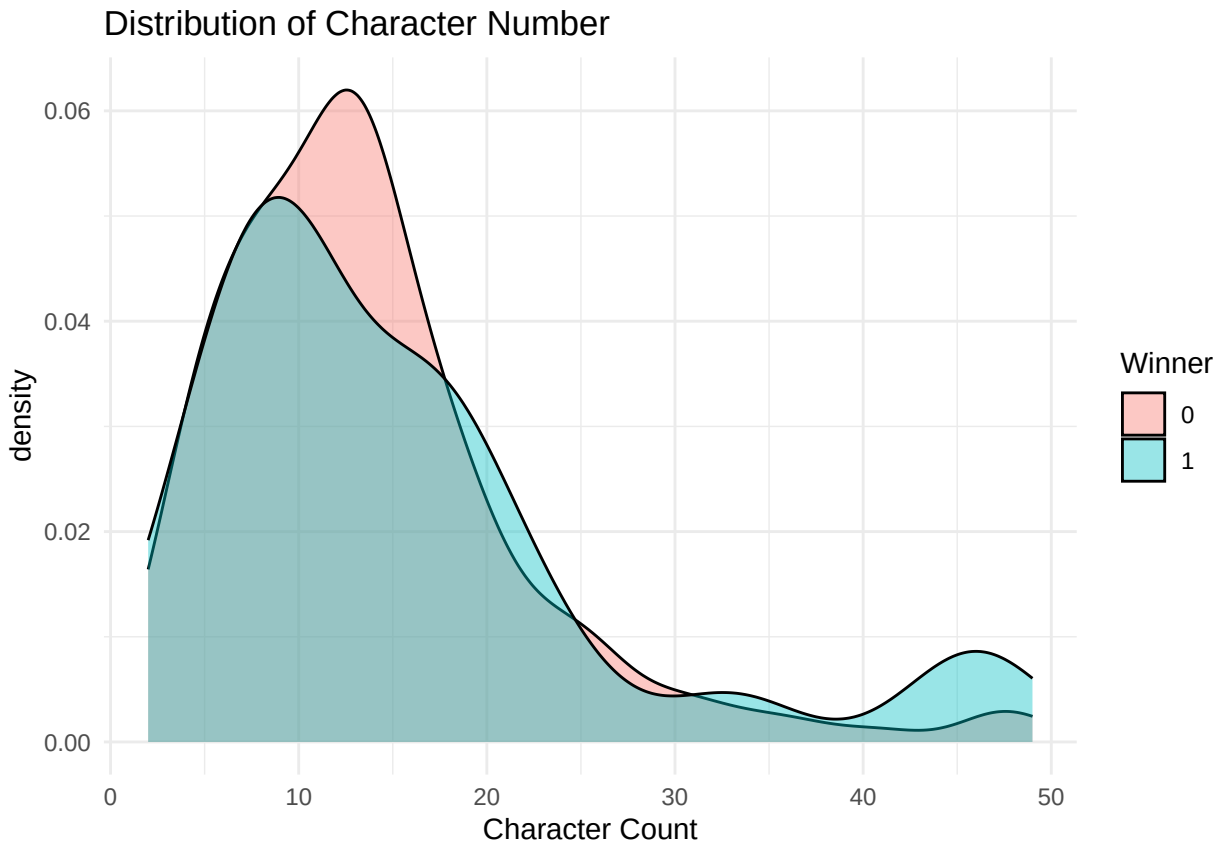
```
# D. The 'Epic' Factor: Runtime
ggplot(d, aes(x = running_time_minutes, fill = factor(best_picture_winner))) + geom_density(alpha = 0.4)
    labs(title = "Distribution of Film Length", x = "Minutes", fill = "Winner") +
    theme_minimal()
```



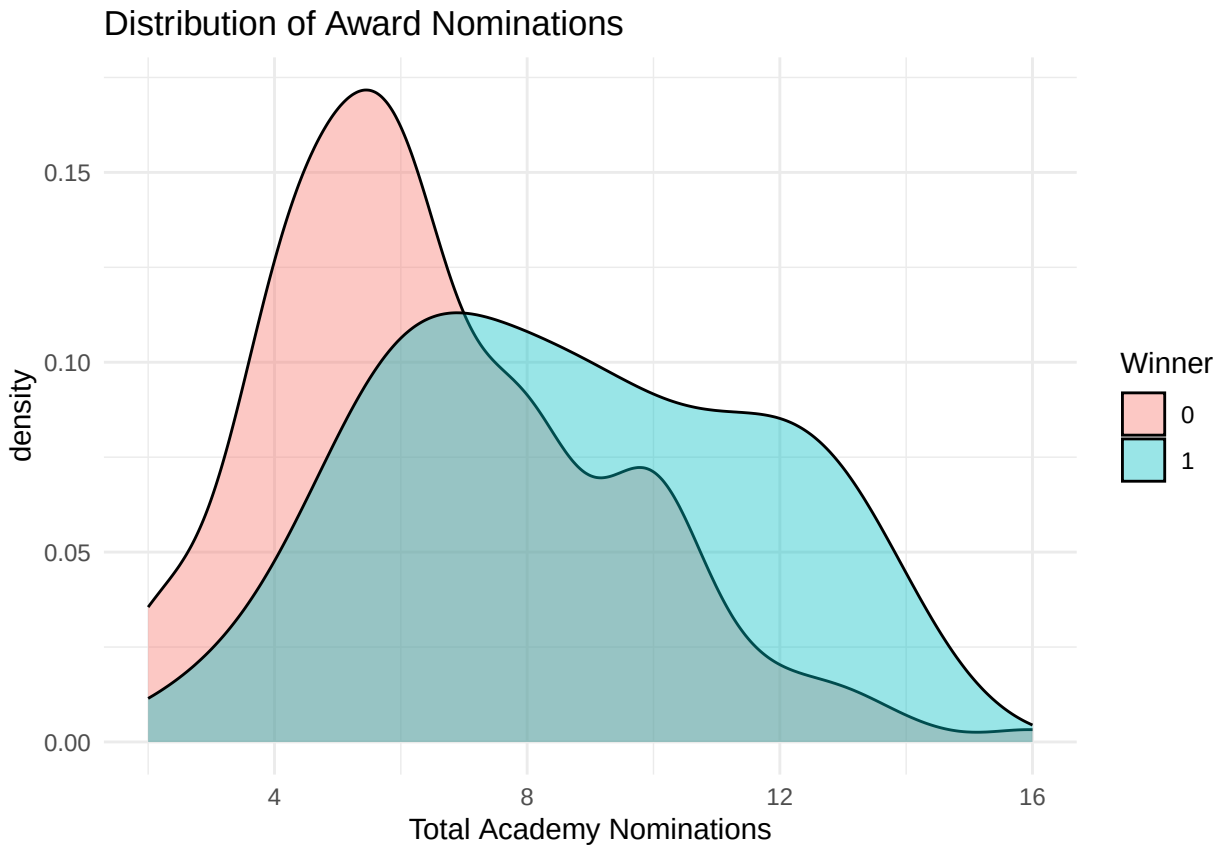

```
# D. The 'Epic' Factor: words count
ggplot(d, aes(x = words, fill = factor(best_picture_winner))) + geom_density(alpha = 0.4) +
    labs(title = "Distribution of Words Count", x = "Word Count", fill = "Winner") +
    theme_minimal()
```



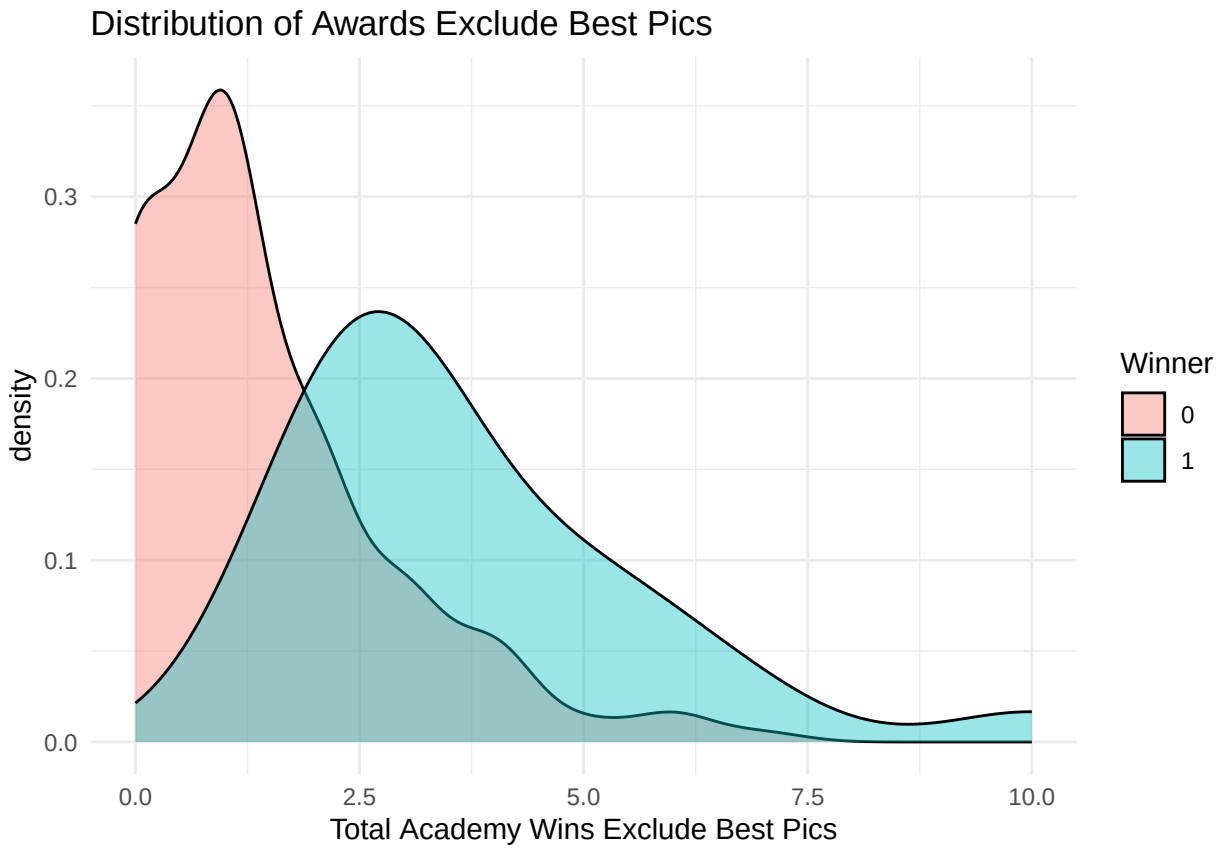

```
# D. The 'Epic' Factor: character count
ggplot(d, aes(x = characters, fill = factor(best_picture_winner))) + geom_density(alpha = 0.4) +
    labs(title = "Distribution of Character Number", x = "Character Count", fill = "Winner") +
    theme_minimal()
```




```
# D. The 'Epic' Factor: character count
ggplot(d, aes(x = total_academy_nominations, fill = factor(best_picture_winner))) +
    geom_density(alpha = 0.4) + labs(title = "Distribution of Award Nominations",
    x = "Total Academy Nominations", fill = "Winner") + theme_minimal()
```




```
# D. The 'Epic' Factor: character count
ggplot(d, aes(x = total_academy_award_exclude_best_pic, fill = factor(best_picture_winner))) +
    geom_density(alpha = 0.4) + labs(title = "Distribution of Awards Exclude Best Pics",
    x = "Total Academy Wins Exclude Best Pics", fill = "Winner") + theme_minimal()
```


Distribution of Awards Exclude Best Pics
density
0.3
0.2
0.1
0.0
0.0
2.5
5.0
7.5
10.0
Total Academy Wins Exclude Best Pics
Winner
0
1